

SCSIT

IV Semester

DataBase

Exc. 8

Cartesian (complex) product
Natural JOIN

Relational algebra Operations

☐ Basic

☐ *Projection*

☐ *Rename*

☐ *Selection*

☐ **Complex product**

☐ Union

☐ Difference

☐ Additional

☐ subtract

☐ ***join - natural***

☐ division

These operation
are binary

Complex product

Tables R and S (could be $R=S$)

R has κ_1 columns = level κ_1 and n_1 rows

S has κ_2 columns = level κ_2 and n_2 rows

General form: $R \times S$ or $R * S$

Result

Table with level $(\kappa_1 + \kappa_2)$ which contains $(n_1 \times n_2)$ rows.

Every row is concatenation form one R row with one S row.

Characteristics

1. If tables have columns with same name, you need to
 - **rename these columns** (at last from one of the tables) or
 - use MARKS ***table_name.column_name***

This will avoid column name repetition (it is not recommended the result to have more than one columns with same names)

2. This relation can be created between more than 2 tables (3,4,...)

Example 1

r

A	B
a	1
b	2

s

C	D	E
a	10	+
b	10	+
b	20	-
c	10	-

r x s

A	B	C	D	E
a	1	a	10	+
a	1	b	10	+
a	1	b	20	-
a	1	c	10	-
b	2	a	10	+
b	2	b	10	+
b	2	b	20	-
b	2	c	10	-

Example 2

T1

↓

A	B
1	A
3	Б
2	T

T2:

C	D
O	7
R	5

T3:

E
T
Q

T1 X T2 X T3:

A	B	C	D	E
1	A	O	7	T
3	Б	O	7	T
2	T	O	7	T
1	A	R	5	T
3	Б	R	5	T
2	T	R	5	T
1	A	O	7	Q
3	Б	O	7	Q
2	T	O	7	Q
1	A	R	5	Q
3	Б	R	5	Q
2	T	R	5	Q

Example 3:

T1=

A	B
1	A

T1 X T2: *(with marks)*

T1.A	T1.B	T2.A	T2.B
1	A	O	7
1	A	R	5

T2=

A	B
O	7
R	5

T1 X T2: *(with renaming)*

A	B	C	D
1	A	O	7
1	A	R	5

T1.A=**A**, T1.B=**B**, T2.A=**C**, T2.B=**D**

SQL for complex product

over T1(A1,B1,C1,...N1) and T2(A2,B2,C2,..M2)

With MARKS

SELECT

T1.A1, T1.B1, T1.C1,...T1.N1,

T2.A2, T2.B2,...T2.M2

FROM *T1, T2*

[order by]

SQL for complex product

over T1(A1,B1,C1,...N1) and T2(A2,B2,C2,..M2)

With renaming

SELECT

T1.A1 as X1, T1.B1 as X2, ...,

T2.A2 as Xk, ...T2.M2 as Xz

FROM *T1, T2*

[order by]

SQL for complex product over T1, T2, T3,....

If you are **absolutely sure** that there aren't any columns with same name, you can use *, i.e.

```
SELECT *  
FROM T1, T2, T3, ...
```

TABLES:

work_place and employer1

	ID_PLace	TITLE	SALARY	SECTOR	PHONE	VEHICLE	LAPTOP
1	1	Manager	33500	Marketing	1	1	1
2	2	DBA	27000	IT	0	0	1
3	3	Analyst	28500	IT	1	0	1
4	4	Developer	25000	IT	0	0	1
5	5	Secretary	12500	Administration	0	0	0
6	6	Hygienist	7000	Administration	0	0	0
7	7	Accountant	10500	Finance	1	0	0
8	8	Treasurer	11500	Finance	1	0	0
9	9	Driver	7500	Service	1	1	0
10	10	Event manager	21500	Marketing	1	1	0
11	11	Guardian	14500	Administration	1	0	0
12	12	Associate	NULL	NULL	0	0	0
13	13	Waiter	1000	Catering	0	0	0

	id_Employee	Name	Profession	id_City
1	1	Mile Mileski	DBA	1
2	2	Maja Sazdova	Treasurer	2
3	3	Aleksandar Popov	Accountant	3
4	4	Gorjan Pehchev	Driver	3
5	5	Emma Petrova	Developer	4
6	6	Kristijan Petrovi.	Manager	1
7	7	Aleksandar Stefanoski	Event manager	1
8	8	Kosta Mickov	Manager	5
9	9	Ana Mircevska	Event manager	1
10	10	Ilina Blagoev	Guardian	10
11	11	Ivo Tomovski	Secretary	1
12	12	Ljupka Ristovska	Accountant	1

Example 4.

SELECT * FROM WORK_PLACE, EMPLOYER

	ID_Place	TITLE	SALARY	SECTOR	PHONE	VEHICLE	LAPTOP	id_Employee	Name	Profession	id_City
1	1	Manager	33500	Marketing	1	1	1	1	Mile Mileski	DBA	1
2	2	DBA	27000	IT	0	0	1	1	Mile Mileski	DBA	1
3	3	Analyst	28500	IT	1	0	1	1	Mile Mileski	DBA	1
4	4	Developer	25000	IT	0	0	1	1	Mile Mileski	DBA	1
5	5	Secretary	12500	Administration	0	0	0	1	Mile Mileski	DBA	1
6	6	Hygienist	7000	Administration	0	0	0	1	Mile Mileski	DBA	1
7	7	Accountant	10500	Finance	1	0	0	1	Mile Mileski	DBA	1
8	8	Treasurer	11500	Finance	1	0	0	1	Mile Mileski	DBA	1
9	9	Driver	7500	Service	1	1	0	1	Mile Mileski	DBA	1
10	10	Event manager	21500	Marketing	1	1	1	1	Mile Mileski	DBA	1
11	11	Guardian	14500	Administration	1	0	0	1	Mile Mileski	DBA	1
12	12	Associate	NULL	NULL	0	0	0	1	Mile Mileski	DBA	1
13	13	Waiter	1000	Catering	0	0	0	1	Mile Mileski	DBA	1
14	1	Manager	33500	Marketing	1	1	1	2	Maja Sazdova	Treasurer	2
15	2	DBA	27000	IT	0	0	1	2	Maja Sazdova	Treasurer	2
16	3	Analyst	28500	IT	1	0	1	2	Maja Sazdova	Treasurer	2
17	4	Developer	25000	IT	0	0	1	2	Maja Sazdova	Treasurer	2
18	5	Secretary	12500	Administration	0	0	0	2	Maja Sazdova	Treasurer	2
19	6	Hygienist	7000	Administration	0	0	0	2	Maja Sazdova	Treasurer	2
20	7	Accountant	10500	Finance	1	0	0	2	Maja Sazdova	Treasurer	2
21	8	Treasurer	11500	Finance	1	0	0	2	Maja Sazdova	Treasurer	2
22	9	Driver	7500	Service	1	1	0	2	Maja Sazdova	Treasurer	2
23	10	Event manager	21500	Marketing	1	1	1	2	Maja Sazdova	Treasurer	2
24	11	Guardian	14500	Administration	1	0	0	2	Maja Sazdova	Treasurer	2

How many ROWS? $156 = 13 * 12$ (13 x 12)

Example 5

```
SELECT  
EMPLOYER.PROFESSION,  
WORK_PLACE.SECTOR,  
EMPLOYER.NAME, WORK_PLACE.SALARY  
FROM WORK_PLACE, EMPLOYER
```

Or, if ALL columns in both table have different name, you can write

```
SELECT  
PROFESSION, SECTOR, NAME, SALARY  
FROM WORK_PLACE, EMPLOYER
```

	PROFESSION	SECTOR	Name	SALARY
1	DBA	Marketing	Mile Mileski	33500
2	Treasurer	Marketing	Maja Sazdova	33500
3	Accountant	Marketing	Aleksandar Popov	33500
4	Driver	Marketing	Gorjan Pehchev	33500
5	Developer	Marketing	Emma Petrova	33500
6	Manager	Marketing	Kristijan Petrovi.	33500
7	Event manager	Marketing	Aleksandar Stefanoski	33500
8	Manager	Marketing	Kosta Mickov	33500
9	Event manager	Marketing	Ana Mircevska	33500
10	Guardian	Marketing	Ilina Blagoev	33500
11	Secretary	Marketing	Ivo Tomovski	33500
12	Accountant	Marketing	Ljupka Ristovska	33500
13	DBA	IT	Mile Mileski	28500
14	Treasurer	IT	Maja Sazdova	28500
15	Accountant	IT	Aleksandar Popov	28500
16	Driver	IT	Gorjan Pehchev	28500
17	Developer	IT	Emma Petrova	28500
18	Manager	IT	Kristijan Petrovi.	28500
19	Event manager	IT	Aleksandar Stefanoski	28500
20	Manager	IT	Kosta Mickov	28500
21	Event manager	IT	Ana Mircevska	28500
22	Guardian	IT	Ilina Blagoev	28500
23	Secretary	IT	Ivo Tomovski	28500
24	Accountant	IT	Ljupka Ristovska	28500

Do we have usage from this result set?

It is not possible **Mile to work as DBA in IT and in the same time to work in all sectors (as DBA)**, when you know that DBA is an IT sector working place?!

Same is for every other worker.

How many ROWS do you have in the result set?

No. of rows in EMPLOYER * No. of rows in work_place

Conclusion:

Complex product does not give distilled (clean) data.

There has to be additional CONDITION for data clean up.

Ex.

SELECT *

FROM WORK_PLACE, EMPLOYER1

WHERE

EMPLOYER1.PROFESSION=WORK_PLACE.TITLE

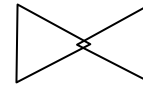
ID_PLACE	TITLE	SALARY	SECTOR	PHONE	VEHICLE	LAPTOP	id_EMPLOYER	NAME_EMP	PROFESSION	C_ID_CITY
1	MANAGER	33500.0	MARKETING	1	1	1	6	Kristijan Petrovik	MANAGER	1
1	MANAGER	33500.0	MARKETING	1	1	1	8	Kosta Mickovski	MANAGER	5
2	DBA	27000.0	IT	0	0	1	1	Mile Mileski	DBA	1
4	PROGRAMMER	25000.0	IT	0	0	1	5	Ema Petrov	PROGRAMMER	4
5	SECRETARY	12500.0	ADMINISTRATION	0	0	0	11	Ivo Tomovski	SECRETARY	1
6	DOORMAN	7000.0	ADMINISTRATION	0	0	0	10	Ilin Blagoev	DOORMAN	10
7	ACCOUNTANT	10500.0	FINANCE	1	0	0	3	Aleksandar Popov	ACCOUNTANT	3
7	ACCOUNTANT	10500.0	FINANCE	1	0	0	12	Милка Ристовска	ACCOUNTANT	1
8	TREASURER	11500.0	FINANCE	1	0	0	2	Maja Sazdova	TREASURER	2
9	DRIVER	7500.0	SERVICE	1	1	0	4	Gorjan Pehcev	DRIVER	3

Now, this is good result:

Now you can see the salary per worker, although in table employer, this data does not exist.

With equalization of column **PROFESSION** from employer and **TITLE** from work_place, we made joining of adequate data from both tables.

JOIN - θ



Complex product with condition

General form:

$$R \theta S = R \bowtie_F S - F\text{-condition}$$

$\sigma_F(R \bowtie S)$ – JOIN thorough Selection

Characteristics:

1. Commutative operation:
2. The result contains rows from $R \bowtie S$ that satisfy the F condition

Example 6.

r

A	B	C
1	2	3
4	5	6
7	8	9

s

D	E
3	1
6	2
1	8

r ⋈_{B < D} s

A	B	C	D	E
1	2	3	3	1
1	2	3	6	2
4	5	6	6	2

SQL for NATURAL JOIN

1. SELECT columns from *table1 and/or table2*
FROM table1, table2

WHERE

table1.column***I*** ***operator for comparison*** table2.column***J***

[additional conditions]

[group by]

[order by]

...

table1.column***I*** ***and*** table2.column***J***

MUST HAVE SAME DATA TYPE

....

operator for comparison : =, >, <, >=, <=, !=, <>, like ...

SQL for NATURAL JOIN

2.1.

SELECT columns from *table1* and/or *table2*

FROM *table1* **JOIN** *table2*

ON

table1.columnI operator for comparison table2.columnJ

[**WHERE** additional conditions]

[group by]

[order by]

*table1.columnI and table2.columnJ
MUST HAVE SAME DATA TYPE*

...

....

SQL for NATURAL JOIN

2.2.

SELECT columns from *table1 and/or table2*
FROM table1 **INNER JOIN** table2

ON

table1.column***I*** *operator for comparison* table2.column***J***
[WHERE additional conditions]

[group by]

[order by]

table1.column***I*** ***and*** table2.column***J***
MUST HAVE SAME DATA TYPE

...

....

SQL for Ex. 6

```
SELECT *  
FROM R,S  
WHERE B<D
```

```
SELECT *  
FROM R [INNER] JOIN S  
ON B<D
```

Ex.7 done with first SQL

- Find all data for employers and their work places, according to the work place title.

1.

```
SELECT *
```

```
FROM WORK_PLACE, EMPLOYER1
```

```
WHERE
```

```
EMPLOYER1.PROFESSION=WORK_PLACE.TITLE
```

Ex. 7 with [INNER] JOIN

- Find all data for employers and their work places, according to the work place title.

2.

SELECT *

FROM WORK_PLACE [INNER] JOIN EMPLOYER1

ON EMPLOYER1.PROFESSION=WORK_PLACE.TITLE

Write 2 types of SQL statements for the following requests

1. Find name of the employer, its profession and salary for those employees that have salary over 10000

2. Find name of the employer and salary for those employees that are either DBA or they work in MARKETING (sector)
 1. List the sector, too.

Write 2 types of SQL statements for the following requests

3. Find the name of the employer that has maximal salary.

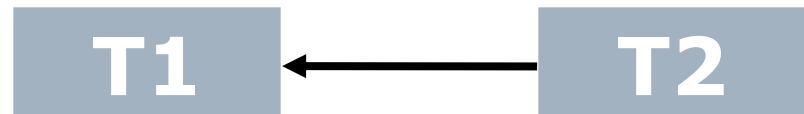
4. Find the name of the emp. and their profession for those emp. That don't work in sector whose employees have laptops.
 1. list its/their salary and sector

Where to use JOIN?

SQL for 1-n related Tables, i.e. Tables with FOREIGN KEYS.

T1(K1 **PK**, k2, **K3** **FK**)

T2(M1 **PK**, M2)



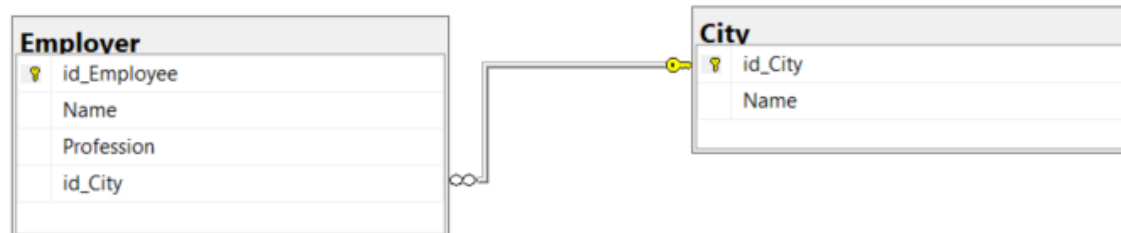
JOIN condition is equalization of **T2.M1** and **T1.K3**
because **T1.K3** is **foreign key in T1** which refers to
T2 primary key (**M1**)

Tables: CITY and EMPLOYER

	id_City	Name
1	1	Skopje
2	2	Ohrid
3	3	Bitola
4	4	Prilep
5	5	Veles
6	6	Tetovo
7	7	Gostivar
8	8	Kumanovo
9	9	Debar
10	10	Kichevo
11	11	Krushevo
12	12	Struga
13	13	Resen
14	14	Negotino
15	15	Kavadarci
16	16	Gevgelija
17	17	Strumica
18	18	Stip
19	19	Radovich
20	20	Kochani
21	21	Kratovo
22	22	Probistip
23	23	Berovo
24	24	Pehchevo
25	25	Sveti Nikole
26	26	Delchevo
27	27	Valandovo
28	28	Kriva Palanka
29	29	Makedonski Brod

Click to

	id_Employee	Name	Profession	id_City
1	1	Mile Mileski	DBA	1
2	2	Maja Sazdova	Treasurer	2
3	3	Aleksandar Popov	Accountant	3
4	4	Gorjan Pehchev	Driver	3
5	5	Emma Petrova	Developer	4
6	6	Kristijan Petrovi.	Manager	1
7	7	Aleksandar Stefanoski	Event manager	1
8	8	Kosta Mickov	Manager	5
9	9	Ana Mircevska	Event manager	1
10	10	Ilina Blagoev	Guardian	10
11	11	Ivo Tomovski	Secretary	1
12	12	Ljupka Ristovska	Accountant	1



EMPLOYER1.C_ID_CITY is FK for **CITY.ID_CITY**

IMPORTANT !!!

- ☐ Every time, **before creating SQL query**,
 - Write down
 - ☐ **Columns** that are asked to be part of the result set
 - ☐ **Tables** these columns are part of
 - **Check** weather there are *some relations between these tables and columns* (P.K – F.K relation or any other way) so that you can create JOUN condition. If there is any, write it down
 - Write down the additional conditions (if any)
- ☐ **NOW, create SQL query**

FOR EVERY Request,
write two types of SQL
Queries

(“=” and

[INNER] JOIN)

5. Find which worker comes from which city

```
SELECT *
FROM EMPLOYER, CITY
WHERE EMPLOYER.ID_CITY= CITY.ID_CITY
```

```
SELECT *
FROM EMPLOYER INNER JOIN CITY
ON EMPLOYER.ID_CITY= CITY.ID_CITY
```

	id_EMPLOYER	NAME_EMP	PROFESSION	C_ID_CITY	id_CITY	CITY_NAME
1	1	Mile Mileski	DBA	1	1	Skopje
2	2	Maja Sazdova	TREASURER	2	2	Ohrid
3	3	Aleksandar Popov	ACCOUNTANT	3	3	Bitola
4	4	Gorjan Pehcev	DRIVER	3	3	Bitola
5	5	Ema Petrov	PROGRAMMER	4	4	Prilep
6	6	Kristijan Petrovik	MANAGER	1	1	Skopje
7	7	Aleksandar Stefanoski	Event manager	1	1	Skopje
8	8	Kosta Mickovski	MANAGER	5	5	Veles

6. Use sql from 5 and find the workers name and city name.

	NAME_EMP	CITY_NAME
1	Mile Mileski	Skopje
2	Maja Sazdova	Ohrid
3	Aleksandar Popov	Bitola
4	Gorjan Pehcev	Bitola
5	Ema Petrov	Prilep
6	Kristijan Petrovik	Skopje
7	Aleksandar Stefanoski	Skopje
8	Kosta Mickovski	Veles

7. Find the workers name and name of the city they're coming from, for those who work as Accountant

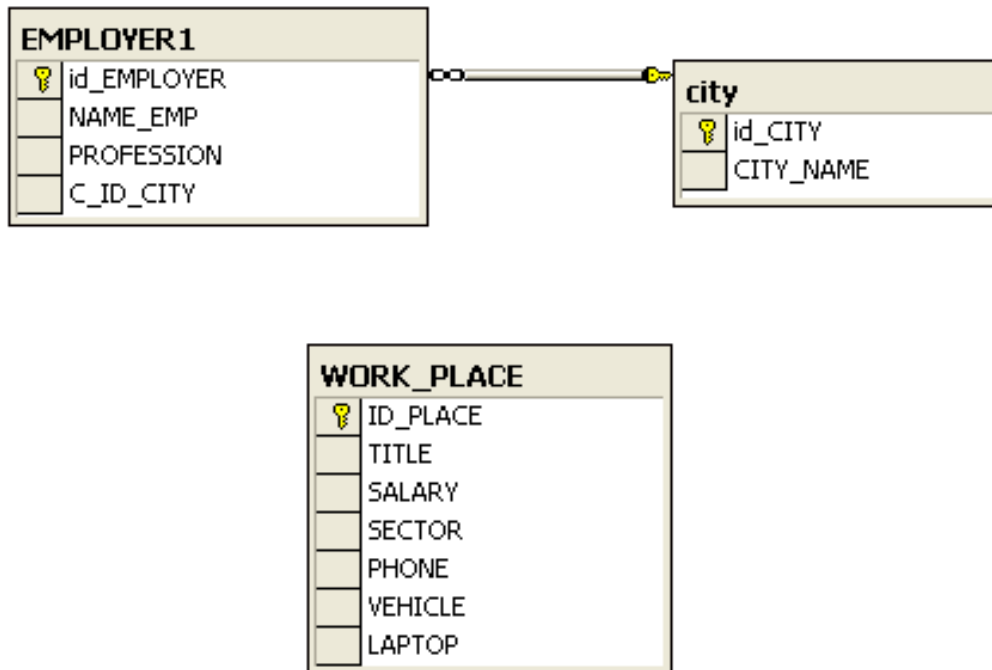
8. Find worker names and working places for those who live in Skopje.

**9. Find the names of the workers that work as
“Driver” or live in Bitola**

**10. Find the name and working place for people
that don't live in Kumanovo or Ohrid**

11. Find which worker comes from which town and works in which sector.

List all data



Although employer1 & Work_place are not connected with primary/foreign key relation, the **columns** **rabotnik1.profession** and **work_place.title** have same domain, so we can use them for equalization.



SELECT *

FROM EMPLOYER, WORK_PLACE, CITY

WHERE EMPLOYER.ID_CITY=CITY.ID_CITY AND

EMPLOYER.PROFESSION= WORK_PLACE.TITLE

	id_...	NAME_EMP	PROFESSION	C_I...	ID...	TITLE	SALARY	SECTOR	PHONE	VEHICLE	LAPTOP	i...	CITY_NAME
1	1	Mile Mileski	DBA	1	2	DBA	27000.0	IT	0	0	1	1	Skopje
2	2	Maja Sazdova	TREASURER	2	8	TREASURER	11500.0	FINANCE	1	0	0	2	Ohrid
3	3	Aleksandar ...	ACCOUNTANT	3	7	ACCOUNTANT	10500.0	FINANCE	1	0	0	3	Bitola
4	4	Gorjan Pehcev	DRIVER	3	9	DRIVER	7500.0	SERVICE	1	1	0	3	Bitola
5	5	Ema Petrov	PROGRAMMER	4	4	PROGRAMMER	25000.0	IT	0	0	1	4	Prilep
6	6	Kristijan P...	MANAGER	1	1	MANAGER	33500.0	MARKETING	1	1	1	1	Skopje
7	8	Kosta Mickovski	MANAGER	5	1	MANAGER	33500.0	MARKETING	1	1	1	5	Veles
8	10	Ilin Blagoev	DOORMAN	10	6	DOORMAN	7000.0	ADMINI...	0	0	0	10	Kicevo

SELECT *

FROM EMPLOYER INNER JOIN CITY

ON EMPLOYER.ID_CITY=CITY.ID_CITY

INNER JOIN WORK_PLACE

ON EMPLOYER.PROFESSION= WORK_PLACE.TITLE

12. Find worker name, city name, sector and salary

Order the data by salary (highest to smallest)

13. Find all workers and cities they're coming from for those in IT sector.

Find working place names

14. Find all workers and their salaries for those who live in Skopje and work in FINANCE sector

15. Find all sectors that have workers from Bitola

16. Find salaries for workers that don't live in Skopje

17. Find what every worker owns (laptop, phone, car)